

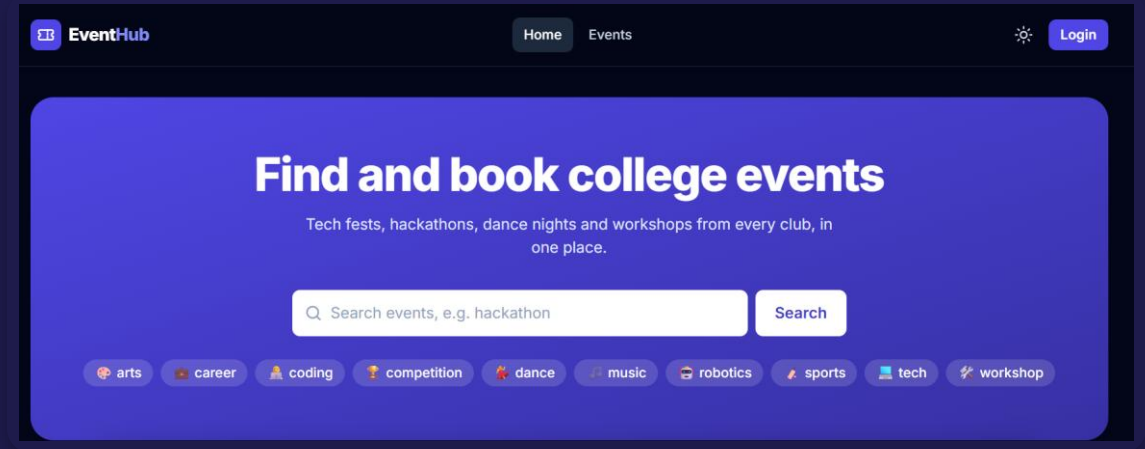


EventHub

College Event Management Platform

Find, book, pay, enter with a QR ticket, get a certificate - all in one place.

Java Full Stack project · React + Spring Boot + MySQL
Madhavan Suresh · September 2026



The problem

Today most college events run on WhatsApp groups and Google Forms



Overbooking

Two students fill the form for the last seat - both think they are in.



Fake entries

A screenshot of a ticket is shared; nobody can tell who already entered.



Late certificates

Made by hand weeks later, and easy to fake on a resume.



No insight

Organizers cannot see sales, attendance or what students thought.

The solution

EventHub puts the whole journey in one place

- Find
- Book and pay (seat held 10 minutes)
- Enter with a one-time QR ticket
- Get a verifiable PDF certificate
- Give feedback - organizers see analytics

Objectives

What the project had to achieve



Easy discovery

Search and filter events by text, tag, club and date - on the server, with paging.



Fair booking

Never sell more seats than exist, even when 100 students click at the same second.



Smart waitlist

Freed seats are offered automatically to the next student in the queue.



Fast, safe entry

QR tickets checked at the gate; each ticket works exactly once.



Proof of attendance

PDF certificates only for students who came, verifiable by anyone.



Insight

Ratings, check-in rate and sales charts for organizers and the admin.

Who uses EventHub

Five kinds of users - the server checks the role on every request



Visitor

- Browse and search events
- Verify a certificate
- Register



Student

- Book and pay
- QR tickets, waitlist
- Certificates, feedback



Organizer

- Create events for their club
- Submit for approval
- Analytics, feedback



Volunteer

- Gate duty for their club
- Scan QR tickets
- Live check-in counter



Admin

- Approve or send back events
- All-club analytics
- Activity (audit) log

The project at a glance

Built step by step in 10 phases, with every change tested on GitHub

29

pages (routes)

50

REST API endpoints

17

database tables

146

automated tests

91%

service test coverage

60+

Git commits

Technologies used

Current, industry-standard tools



Frontend

- React 19 + Vite 8
- Tailwind CSS 4 (dark mode)
- Redux Toolkit
- React Router 7
- Recharts (charts)
- html5-qrcode (camera)



Backend

- Java 25, Spring Boot 4.1
- Spring Security + JWT
- Spring Data JPA / Hibernate
- Flyway migrations
- ZXing (QR), OpenPDF
- Caffeine cache, Spring Mail



Data & services

- MySQL 8.4
- Docker Compose
- Mailpit (test email)
- Stripe Checkout (test mode)
- Swagger / OpenAPI docs

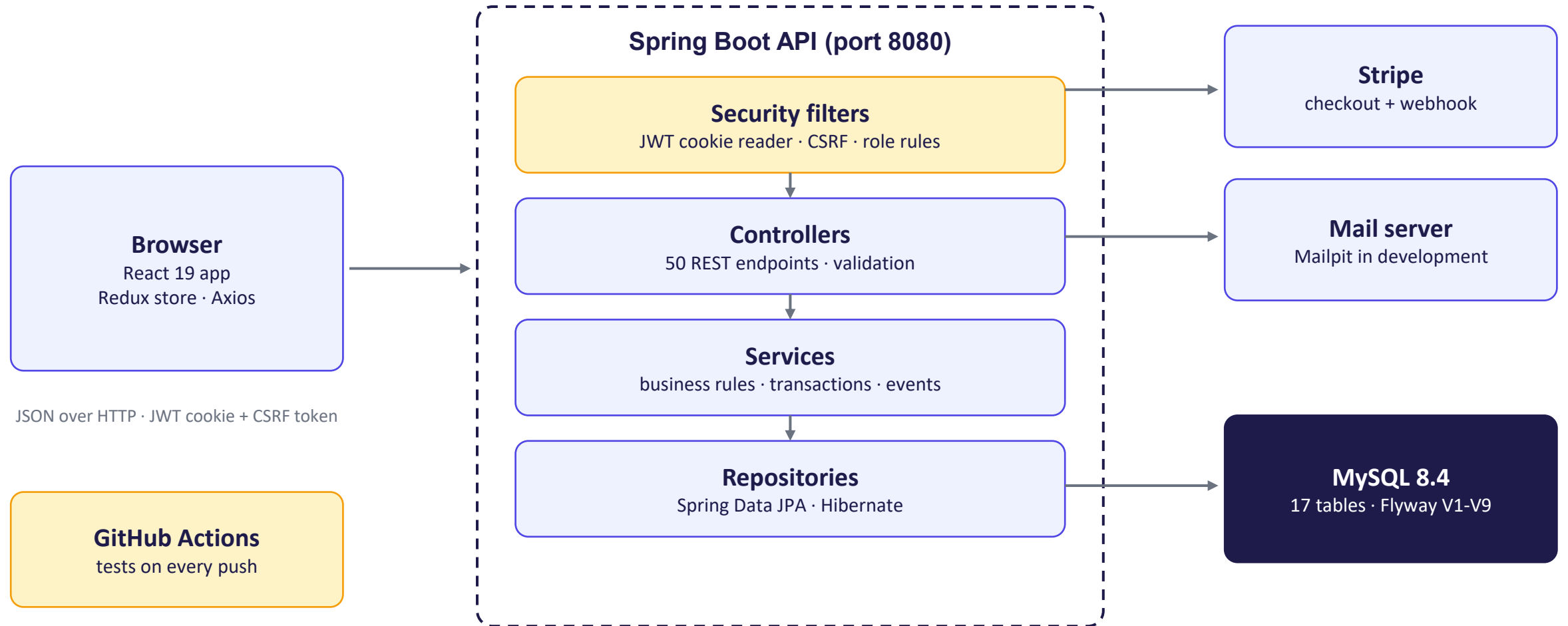


Quality & tools

- JUnit 5 + Mockito
- Testcontainers (real MySQL)
- JaCoCo coverage
- Vitest + Testing Library
- GitHub Actions CI
- Postman, IntelliJ, Git

Architecture

A single-page React app talks to a layered Spring Boot API



Backend modules

One Java package per business area - easy to find and to test

event

Create, edit, search, approval workflow

booking

Seat holds, confirm, cancel, expiry job

payment

Stripe or test gateway, webhooks, refunds

waitlist

Queue, 30-minute seat offers

notification

In-app bell, emails, reminders

gate

QR images, check-in, live counter

certificate

PDF certificates, public verify

feedback

Stars and anonymous comments

analytics

Charts data, 60-second cache

auth

Register, login, JWT, club roles

audit

Who did what, and when

club · tag · user

Clubs, tags, users, profiles

Database design

17 MySQL tables, created and changed only through Flyway migrations (V1-V9)



People & clubs

`users, roles, user_roles, profiles, clubs, club_members`



Events

`events, tags, event_tags, audit_log`



Bookings & money

`bookings, payments, processed_payment_events, waitlist_entries`



After the event

`notifications, certificates, feedback`

Design rules that keep the data correct

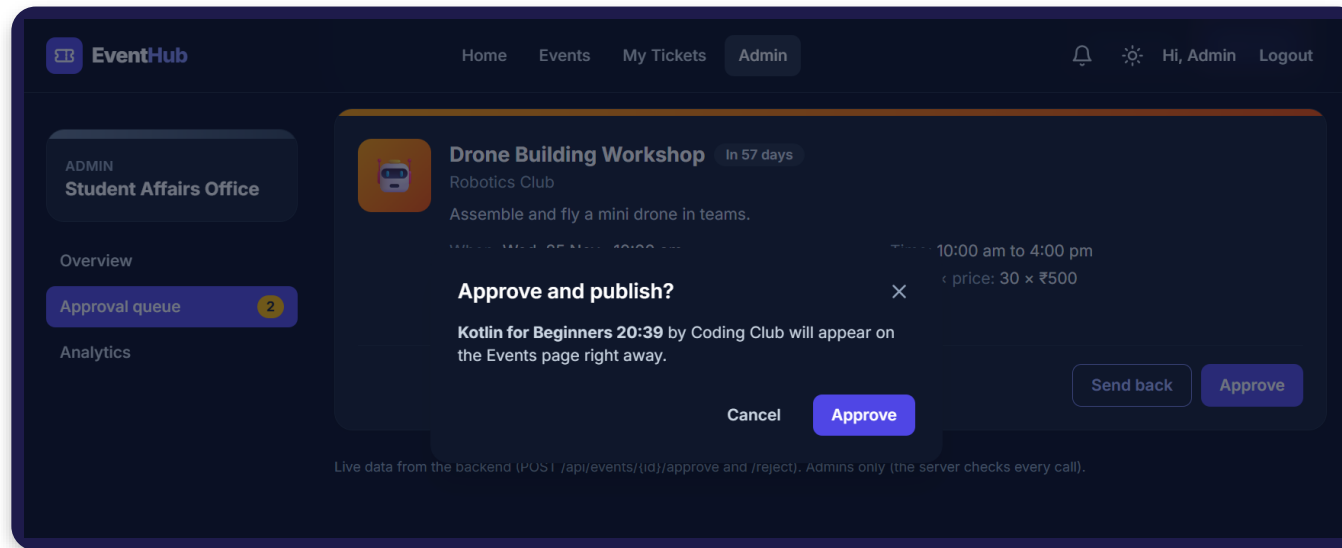
- `events.available_seats` is the ONE seat counter; a `@Version` column stops two saves from overwriting each other
- UNIQUE keys stop duplicates: one certificate and one rating per ticket, one webhook message processed once
- CHECK constraints: status values, 1-5 stars, 1-10 tickets
- Random codes (EVH-..., EH-2026-...) so nobody can guess other tickets or certificates
- Tests run on their own fresh MySQL (Testcontainers)

Workflow 1: an event from idea to published

A small state machine - illegal moves are refused by the server



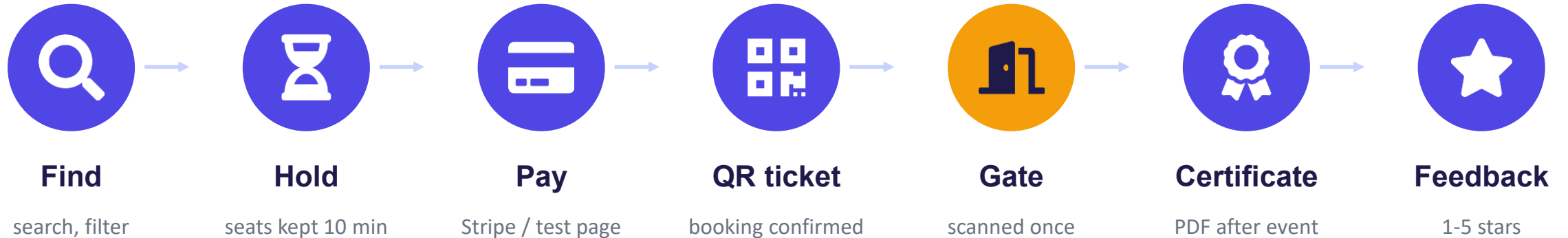
Admin sends back with a reason → back to DRAFT, the organizer sees the note



- Every change is written to the audit log (who, what, when)
- The event row has a version number: two admins clicking at once cannot both win
- Only organizers of THAT club may edit its events

Workflow 2: the student journey

From finding an event to getting a certificate

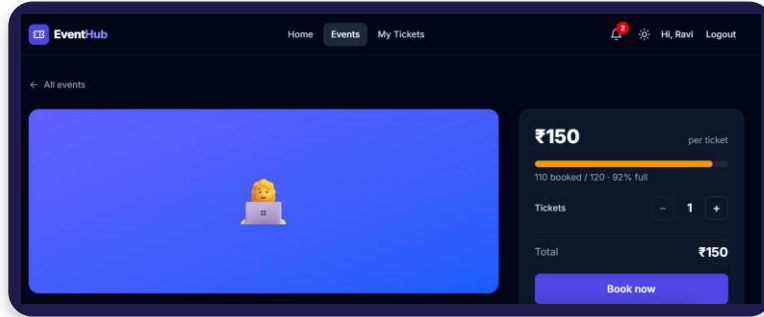


What happens if...

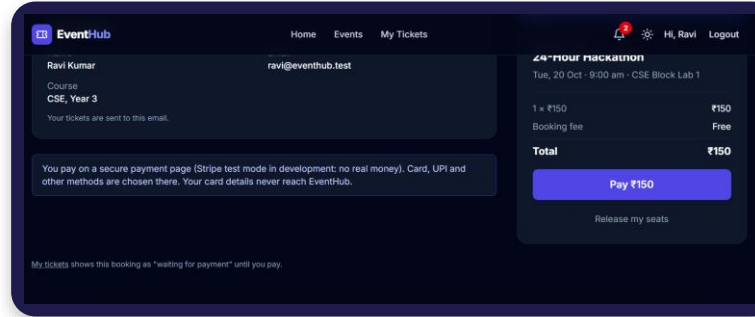
- the event is sold out? → join the waitlist ("You are #3"). When someone cancels, the seats are kept 30 minutes for the next student, who gets an email and a bell notification.
- the student does not pay? → a job running every minute frees the held seats after 10 minutes.
- the payment arrives after the hold ran out? → the seats are taken again if still free, otherwise the money is refunded.

Example 1: Ravi books the 24-Hour Hackathon

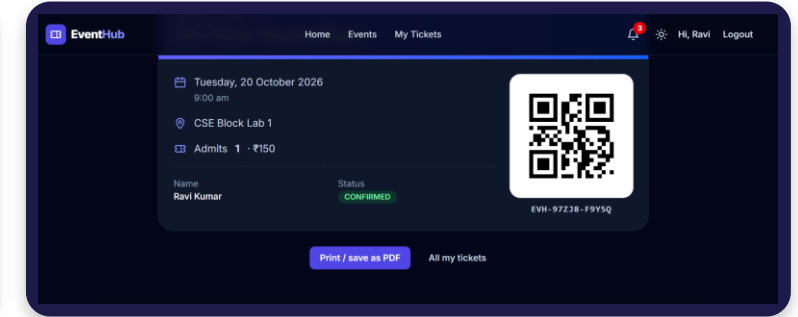
Screens from the real running application



1. Live seat count, choose tickets



2. Seat held - server-side timer



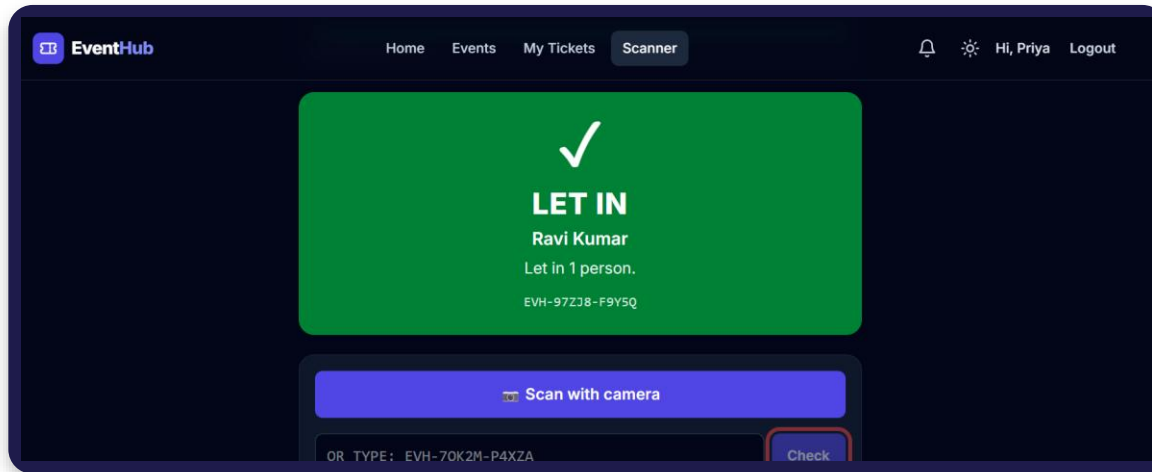
3. Paid → QR ticket

Why two students can never get the same last seat

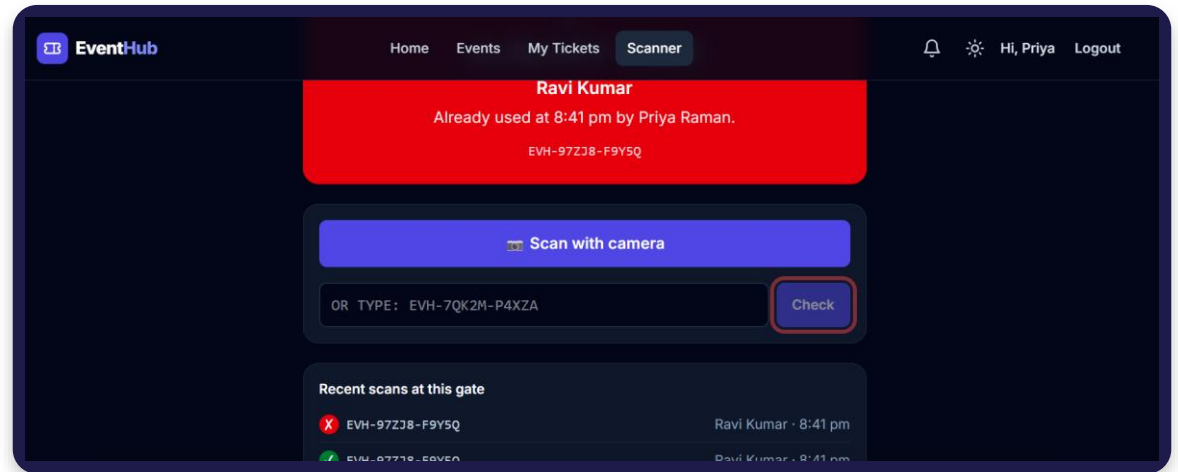
Every save of an event carries its version number. If Ravi and Asha both read "1 seat left, version 7", the first save succeeds (version 8) and the second is refused and retried with fresh data - it now sees 0 seats. A test starts 100 threads for 10 seats: exactly 10 bookings succeed, every time. Without @Version the same test sold 96-99 tickets.

Example 2: the gate - each ticket works once

Volunteer Priya scans Ravi's QR code (camera or typed code)



First scan → LET IN

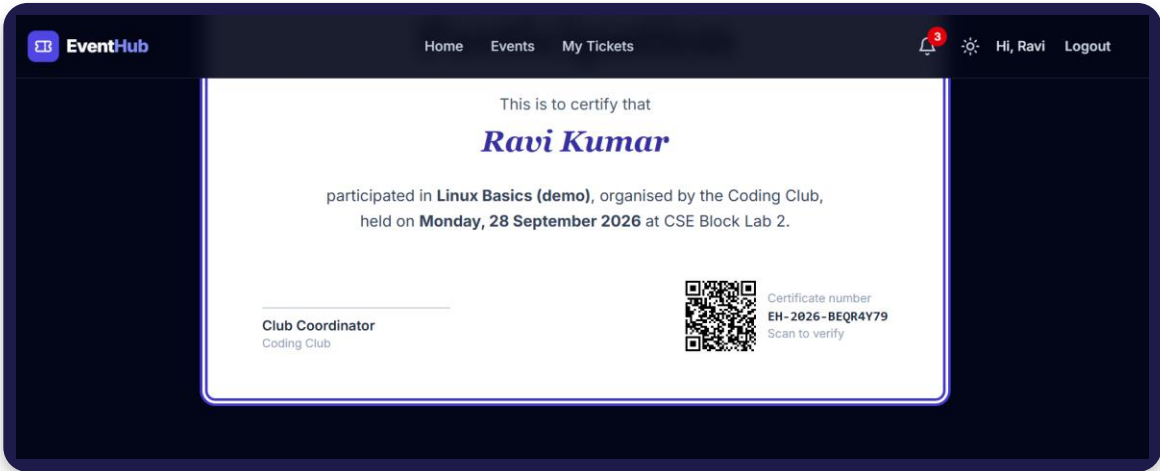


Same ticket again → ALREADY USED (when, by whom)

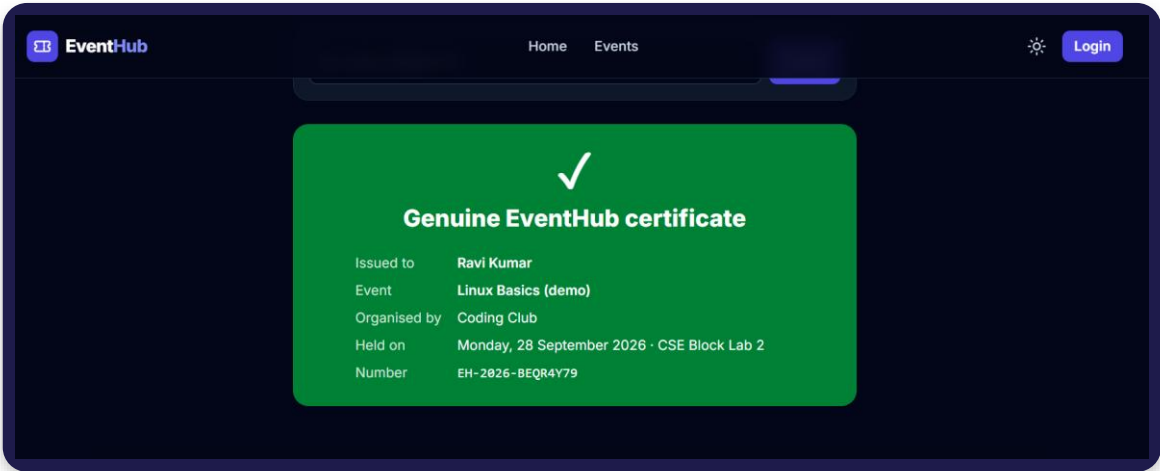
One SQL statement does check-and-mark together: `UPDATE bookings SET checked_in_at = now WHERE id = ? AND checked_in_at IS NULL`. If it changed 1 row → VALID; 0 rows → someone was faster → ALREADY USED. A test with 20 gates scanning the same ticket at the same moment gets exactly one VALID.

Example 3: certificates anyone can verify

Only for students who were scanned at the gate, after the event ends



PDF drawn by the server (OpenPDF), with a QR code



Public verify page - no login needed



Random number EH-2026-BEQR4Y79 - cannot be guessed



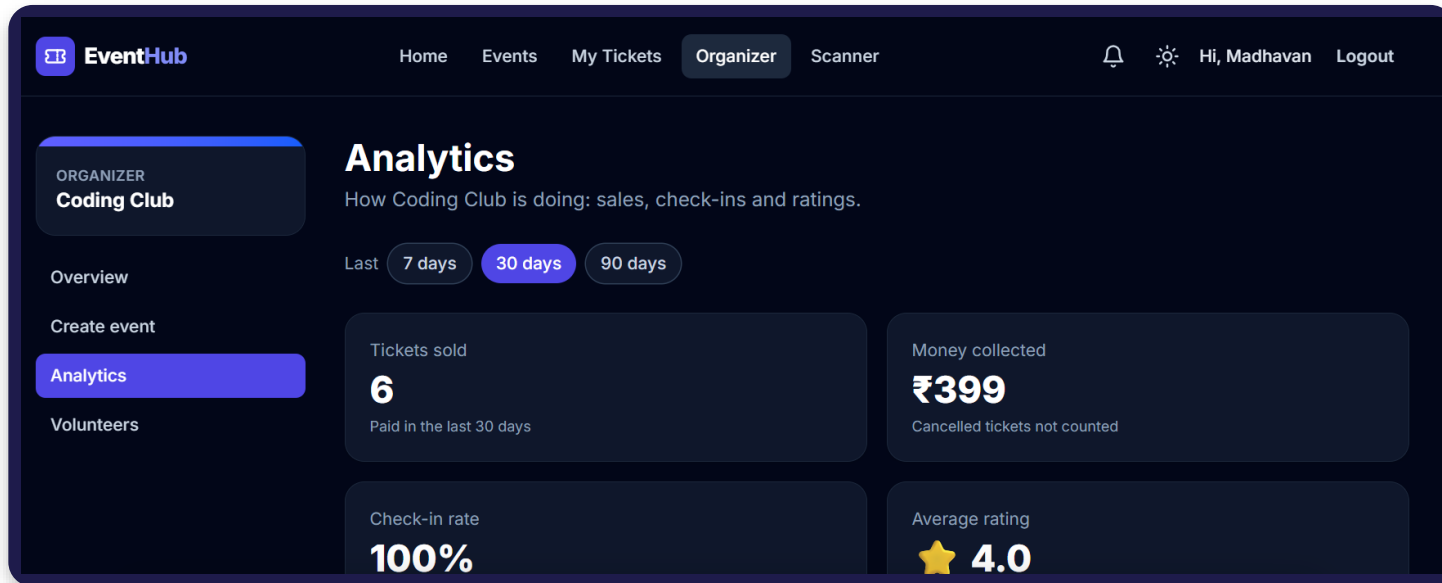
Checked-in + event over = certificate



A company checks it in seconds

Example 4: analytics for organizers and the admin

Tickets and money per day, check-in rate and average rating



Counted in the database

JPQL GROUP BY, SUM and AVG - Java gets a few small rows, not every booking.

Cached for 60 seconds

@Cacheable + Caffeine: repeated views do not touch MySQL. Seat counts are never cached.

Drawn with Recharts

Bar charts with hover tooltips, a table view for screen readers, light and dark colours checked for contrast.

Security

The page only hides buttons - the server protects every action



Passwords hashed with BCrypt
never stored as plain text



JWT in an httpOnly cookie
JavaScript cannot read it, so a script cannot steal it



CSRF protection
other websites cannot send requests as you



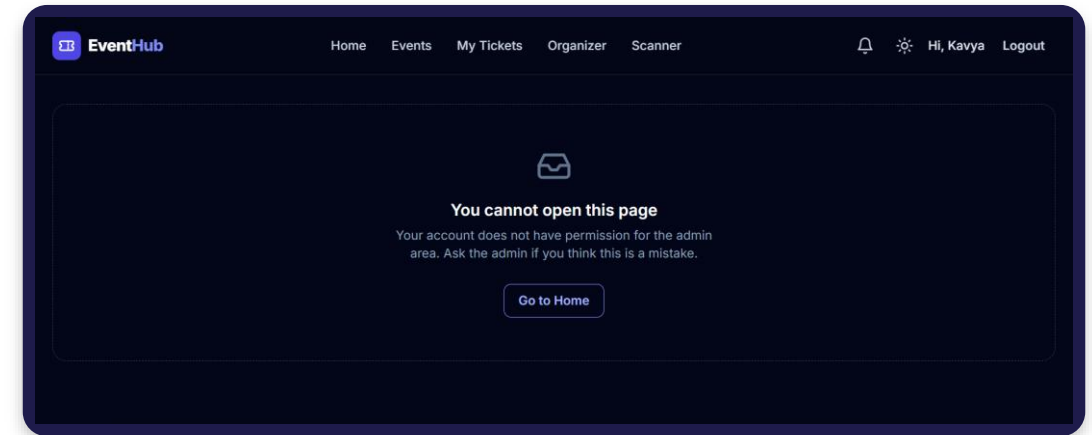
Roles checked on every request
club roles read from the database each time → 403



Login lock
5 wrong passwords → 15 minutes locked (HTTP 429)



Audit log + admin-only tools
Swagger and Actuator for the admin only



*Kavya (organizer of another club) opens the admin area: blocked.
The API answers 403 Forbidden too.*

Testing and quality

Every push to GitHub runs all tests automatically (GitHub Actions)

133

backend tests

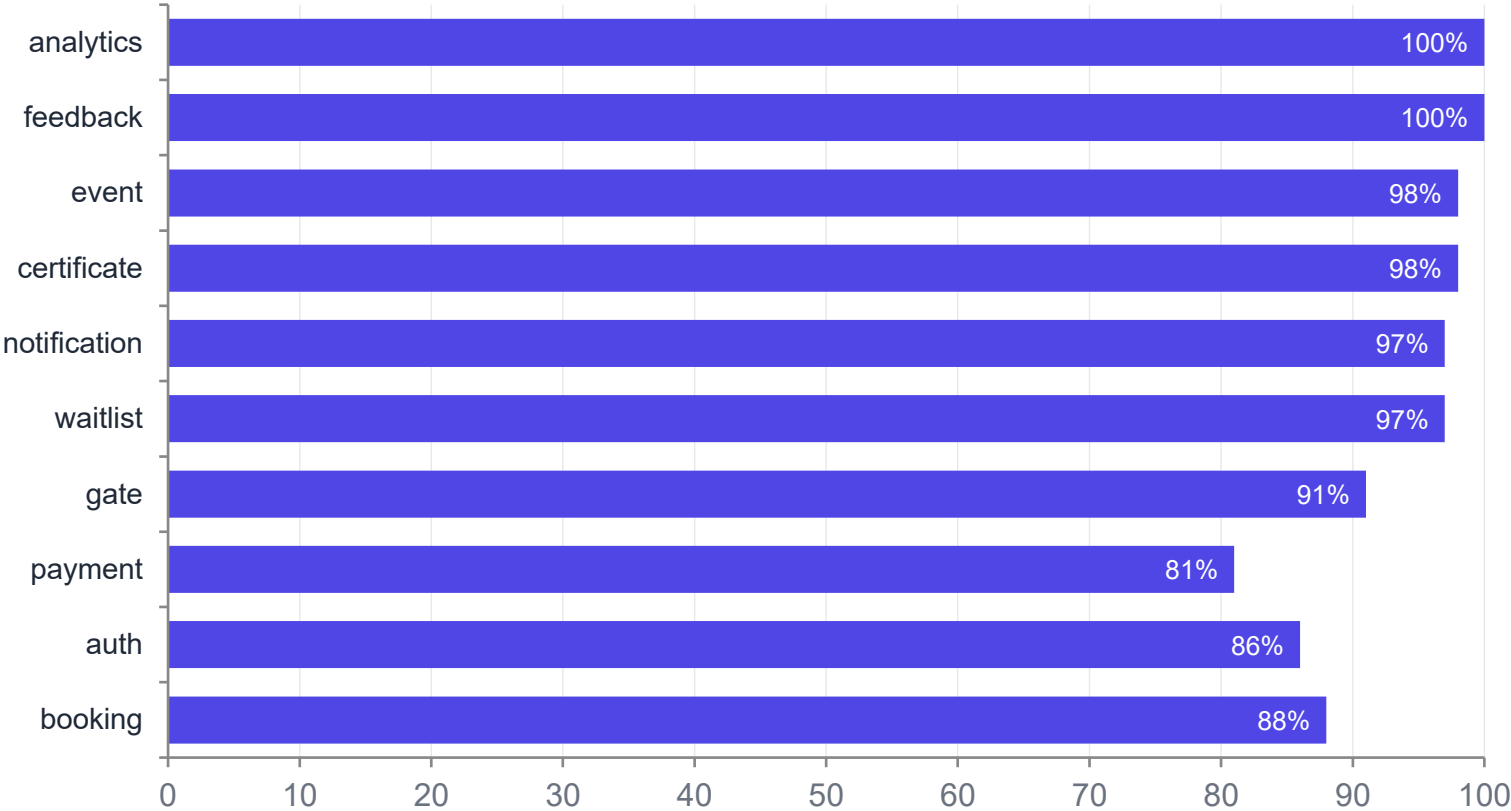
13

React tests

91%

service coverage

Line coverage by module (JaCoCo)



Hard problems solved

The lessons I would explain in an interview

Last seat race

100 students, 10 seats

Fix: Optimistic locking (@Version) + automatic retry

MySQL deadlock

two transactions waiting on each other

Fix: Always update the event row first, then insert the booking

Emails for undone work

mail sent, then the change rolled back

Fix: Save the message in the same transaction; send only after commit

Stale snapshot

REPEATABLE READ kept showing "unused"

Fix: Answer from the UPDATE result instead of reading again

Flaky JWT test

failed 1 run in 16

Fix: The last base64 letter has 2 unused bits - change the first letter

Blank scanner page

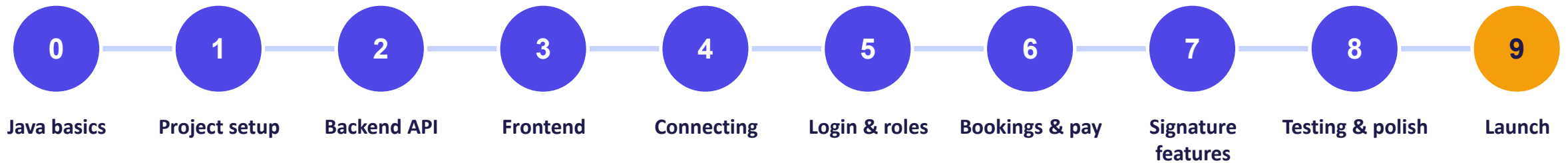
crash after a refactor

Fix: Found by the demo tour; added a page test and a lint rule

How it was built: 10 phases

Each phase ended with "done when" checks before moving on

Phases 0-8 complete · Phase 9 (deployment) next



Working style: small steps, each one tested and committed; every phase started from a written checklist; CI stayed green; a guided demo tour in Edge replays the whole journey (28 steps) and made the screenshots in this deck.

Future scope

What would come next in a real product



Go live

Deploy backend, database and frontend (Phase 9)



Password reset

Reset link by email



Mobile / offline gate

Installable app that scans even with weak Wi-Fi



Indian payments

UPI through Razorpay next to Stripe



More servers

Redis for the login lock and cache



Many colleges

One platform, separate data per college



Conclusion

1

A complete, working full stack product - not just screens: payments, QR entry, PDFs, emails, charts.

2

Correct under pressure: no overselling, no double entry, no emails for undone work.

3

Professional practice: security, 146 automated tests, CI, clean modules and documentation.

Thank you

Questions?

github.com/madhavansuresh000-sys/eventhub

Madhavan Suresh · Java Full Stack